



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



Module

2

SYNTAX ANALYSIS

Dr. B.V. Baiju,
SCOPE, Assistant Professor
VIT, Vellore

SYNTAX ANALYSIS

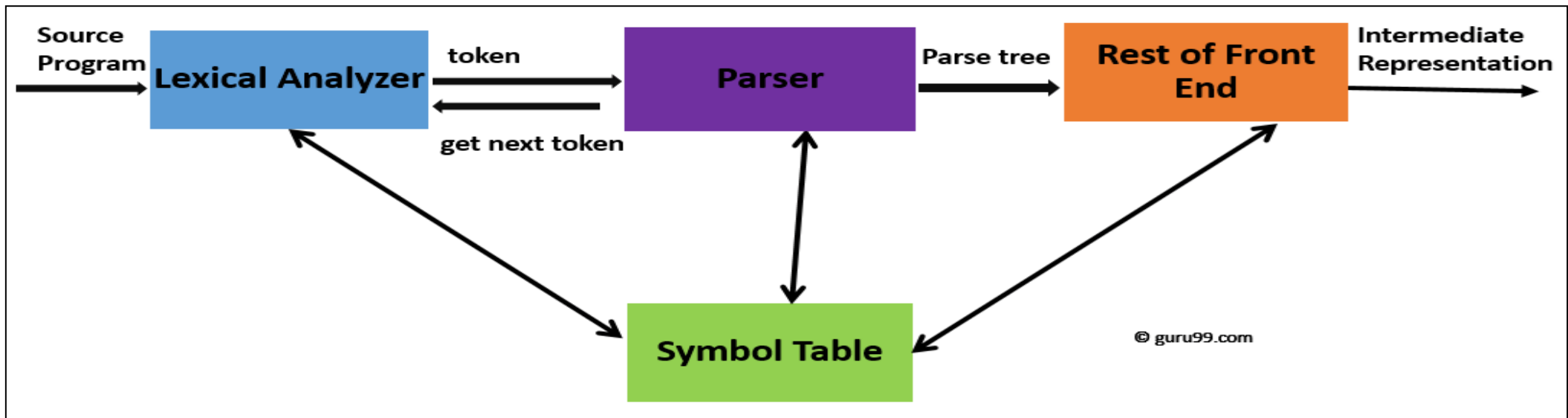
- In syntax analysis, the compiler checks the syntactic structure of the input string, i.e., whether the given string follows the grammar or not.
- It uses a data structure called a parse tree or syntax tree to make comparisons.
- It is used to *check if the code is grammatically correct or not.*
- It helps us to *detect all types of syntax errors.*
- It gives an *exact description of the error.*
- It *rejects invalid code before actual compiling.*

- **Syntax Analyser Terminology**

Sentence	A sentence is a group of character over some alphabet.
Lexeme	A lexeme is the lowest level syntactic unit of a language (e.g., total, start).
Token	A token is just a category of lexemes.
Keywords and reserved words	It is an identifier which is used as a fixed part of the syntax of a statement. It is a reserved word which you can't use as a variable name or identifier.
Noise words	Noise words are optional which are inserted in a statement to enhance the readability of the sentence.
Comments	It is a syntactic element which marks the start or end of some syntactic unit.
Comments	It is a very important part of the documentation. It mostly display by, /* */, or//Blank (spaces)
Character set	ASCII, Unicode
Identifiers	It is a restrictions on the length which helps you to reduce the readability of the sentence.
Operator symbols	+ and – performs two basic arithmetic operations.

ROLE OF THE PARSER – PARSE TREE

- The process of *transforming the data from one format* to another is called Parsing.
- This process can be accomplished by the parser.
- *The parser or syntactic analyzer obtains a string of tokens from the lexical analyzer and verifies that the string can be generated by the grammar for the source language.*
- It reports any syntax errors in the program.
- It also recovers from commonly occurring errors so that it can continue processing its input.



- The tasks of parser can be expressed as
 - Parser *builds the parse tree*.
 - Parser *verifies the structure generated by the tokens* based on the grammar (performs context free syntax analysis).
 - Parser helps to *construct intermediate code*.
 - Parser *produces appropriate error messages*.
 - Parser *performs error recovery*.

Issues:

- Parser cannot detect errors such as:
 - Variable re-declaration
 - Variable initialization before use
 - Data type mismatch for an operation

```
var x = 10;  
console.log(x) // 10  
// Reassign  
var x = 20;  
console.log(x) // 20  
// Redeclare  
x = 30;  
console.log(x) // 30
```

Syntax error handling

- Planning the error handling right from the start can both simplify the structure of a compiler and improve its response to errors.
- Programs can contain errors at many different levels.
- For example :
 1. **Lexical** - such as misspelling an identifier, keyword or operator.
 2. **Syntax** - such as misplaced semicolons or extra or missing braces, a case statement without an enclosing switch statement.
 3. **Semantic** - such as type mismatches between operators and operands.
 4. **Logical** - such as an assignment operator = instead of the comparison operator ==, infinitely recursive call

- The error handler in a parser has simple goals:
 - It should report the presence of errors clearly and accurately.
 - It should recover from each error quickly enough to be able to detect subsequent errors.
 - It should not significantly slow down the processing of correct programs

Error recovery strategies :

- The different strategies that a parser uses to recover from a syntactic error are:
 1. Panic mode
 2. Phrase level
 3. Error productions
 4. Global correction

1. Panic-mode:

- In the case when the parser encounters an error, this mode ignores the rest of the statement and not process input from incorrect input to delimiter, like a semi-colon.
- This is a simple error recovery method.
- In this type of recovery method, the parser rejects input symbols one by one until a single designated group of synchronizing tokens is found.
- The synchronizing tokens generally using delimiters like ';' or end.

2. Phrase-level:

- Compiler corrects the program by inserting or deleting tokens.
- This allows it to proceed to parse from where it was.
- It performs correction on the remaining input.
- It can replace a prefix of the remaining input with some string this helps the parser to continue the process.
- A typical local correction is to replace a comma by a semicolon, delete an extraneous semicolon, or insert a missing semicolon.

3. Error productions:

- Error production recovery expands the grammar for the language which generates the incorrect constructs.
- The parser then performs error diagnostic about that construct.

4. Global correction:

- The compiler should make less number of changes as possible while processing an incorrect input string.
- Given incorrect input string a and grammar c , algorithms will search for a parse tree for a related string b . Like some insertions, deletions, and modification made of tokens needed to transform a into b is as little as possible.

- A parser is a program that generates a parse tree for the given string, if the string is generated from the underlying grammar.



Context free grammar

- Grammars are used to *systematically describe the syntax of programming language constructs* like expressions and statements.
 - A **context free grammar** (CFG) is a 4-tuple $G = (V, \Sigma, S, P)$ where,
 - V is finite set of *non terminals*,
 - Σ is disjoint finite *set of terminals*,
 - S is an element of V and it's a *start symbol*,
 - P is a finite set *formulas/productions* of the form $A \rightarrow \alpha$ where $A \in V$ and $\alpha \in (V \cup \Sigma)^*$
-

► Nonterminal symbol:

- The name of syntax category of a language, e.g., noun, verb, etc.
- These help to *define the language generated by the grammar.*
- Sometimes called as "*syntactic variables.*"
- *Uppercase letters* early in the alphabet, such as A, B, C
- It is written as a **single capital letter**, or as a **name enclosed between** $\langle \dots \rangle$

▶ Terminal symbol:

- A symbol in the alphabet.
 - Sometimes referred to as "*tokens*."
 - *Lowercase letters* early in the alphabet, such as a, b, c.
 - *Operator symbols* such as +, *, - , / and so on.
 - *Punctuation symbols* such as parentheses, comma, and so on.
 - The *digits* 0 , 1 , . . . , 9.
-

▶ Start symbol:

- First nonterminal symbol of the grammar is called start symbol.
-

▶ Production:

- A production, also called *a rewriting rule*, is a rule of grammar.
- It has the form of

LHS → RHS

A nonterminal symbol → String of terminal and nonterminal symbols

Example: Grammar

Write terminals, non terminals, start symbol, and productions for following grammar.

$$E \rightarrow E O E \mid (E) \mid -E \mid \text{id}$$
$$O \rightarrow + \mid - \mid * \mid / \mid \uparrow$$

Terminals: $\text{id} + - * / \uparrow ()$

Non terminals: E, O

Start symbol: E

Productions: $E \rightarrow E O E \mid (E) \mid -E \mid \text{id}$

$$O \rightarrow + \mid - \mid * \mid / \mid \uparrow$$

Derivation

- A grammar derives strings by beginning with the start symbol and repeatedly replacing a nonterminal by the body of a production for that nonterminal. This sequence of replacements is called **derivation**.
- Derivation is *used to find whether the string belongs to a given grammar or not*.
- Given the grammar:

$$\begin{aligned} \text{exp} &\rightarrow \text{exp op exp} \mid (\text{exp}) \mid \text{number} \\ \text{op} &\rightarrow + \mid - \mid * \end{aligned}$$

- Types of derivations are:
 1. Leftmost derivation
 2. Rightmost derivation

(1) $\text{exp} \Rightarrow \text{exp op exp}$	$[\text{exp} \rightarrow \text{exp op exp}]$
(2) $\Rightarrow \text{exp op number}$	$[\text{exp} \rightarrow \text{number}]$
(3) $\Rightarrow \text{exp} * \text{number}$	$[\text{op} \rightarrow *]$
(4) $\Rightarrow (\text{exp}) * \text{number}$	$[\text{exp} \rightarrow (\text{exp})]$
(5) $\Rightarrow (\text{exp op exp}) * \text{number}$	$[\text{exp} \rightarrow \text{exp op exp}]$
(6) $\Rightarrow (\text{exp op number}) * \text{number}$	$[\text{exp} \rightarrow \text{number}]$
(7) $\Rightarrow (\text{exp} - \text{number}) * \text{number}$	$[\text{op} \rightarrow -]$
(8) $\Rightarrow (\text{number} - \text{number}) * \text{number}$	$[\text{exp} \rightarrow \text{number}]$

Parse Trees

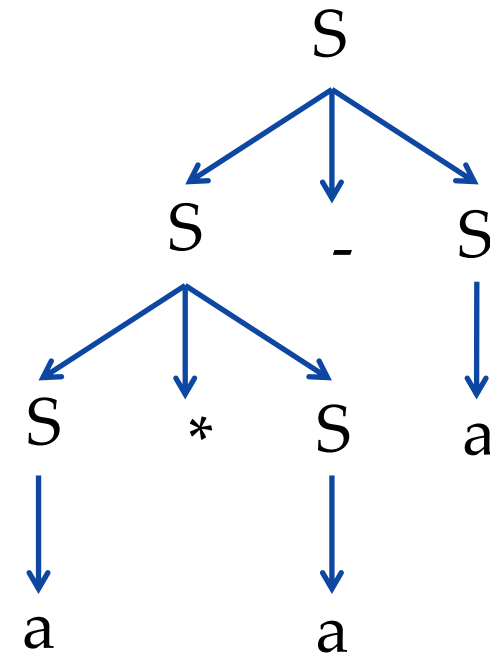
- A parse Tree is a *labeled tree representation of a derivation* that filters out the order in which productions are applied to replace *nonterminals*.
- The **interior nodes** are labeled by *nonterminals*
- The **leaf nodes** are labeled by *terminals*
- The children of each internal node A are labeled, from left to right, by the symbols in the body of the production by which this A was replaced during the derivation.

Leftmost derivation

- A derivation of a string W in a grammar G is a left most derivation if at every step the **left most non terminal** is replaced.
- Grammar:** $S \rightarrow S+S \mid S-S \mid S*S \mid S/S \mid a$ **Output string:** a^*a-a

S
 $\rightarrow$ S-S
 $\rightarrow$ S*S-S
 $\rightarrow$ a*S-S
 $\rightarrow$ a*a-S
 $\rightarrow$ a*a-a

Parse tree represents the
structure of derivation



Parse tree

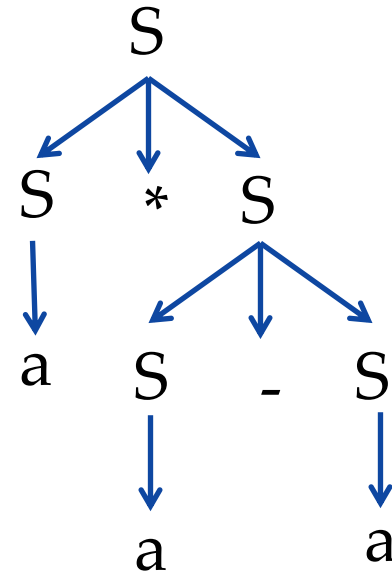
Leftmost Derivation

Rightmost derivation

- A derivation of a string W in a grammar G is a right most derivation if at every step the **right most non terminal** is replaced.
- It is all called **canonical derivation**.
- Grammar: $S \rightarrow S+S \mid S-S \mid S*S \mid S/S \mid a$ Output string: $a*a-a$

$\underline{S}$
 $\rightarrow S*\underline{S}$
 $\rightarrow S*S-\underline{S}$
 $\rightarrow S*S-\underline{a}$
 $\rightarrow S*\underline{a}-a$
 $\rightarrow \underline{a}*a-a$

**Rightmost
Derivation**



Parse Tree

Exercise: Derivation

1. Perform leftmost derivation and rightmost derivation ,and draw parse tree.

$$S \rightarrow A1B$$

$$A \rightarrow 0A \mid \epsilon$$

$$B \rightarrow 0B \mid 1B \mid \epsilon$$

Output string: 1001

2. Perform leftmost derivation and draw parse tree.

$$S \rightarrow 0S1 \mid 01$$

Output string: 000111

3. Perform leftmost and rightmost derivation, and draw parse tree.

$$E \rightarrow E+E \mid E * E \mid \text{id} \mid (E) \mid -E$$

Output string: id + id * id